Feasibility & Implementation Report: A TacticAI-Style Model for the Vertical Stack Offense

Prepared for: BITS Ultimate Frisbee Club application context **Follow-on to:** *Feasibility Report: A TacticAI-Style Tactical AI for Ultimate Frisbee* **Scope:** Narrowing the general Ultimate feasibility report to one concrete, buildable first system — a graph model of the **vertical stack**, the most common continuation offense in the sport.

---

1. Why Vertical Stack Is the Right First Target

The previous report identified the discrete "pull / dead-disc restart" problem as the closest analogue to TacticAI's corner kicks, because it is repeatable, low-variance, and well-labeled. Vertical stack sharpens that further: it is not just a restart, it is a **standing offensive structure with an explicit, named ordering of players** that persists over many consecutive throws. That gives it three properties football set pieces don't have, all of which make the modeling problem easier, not harder:

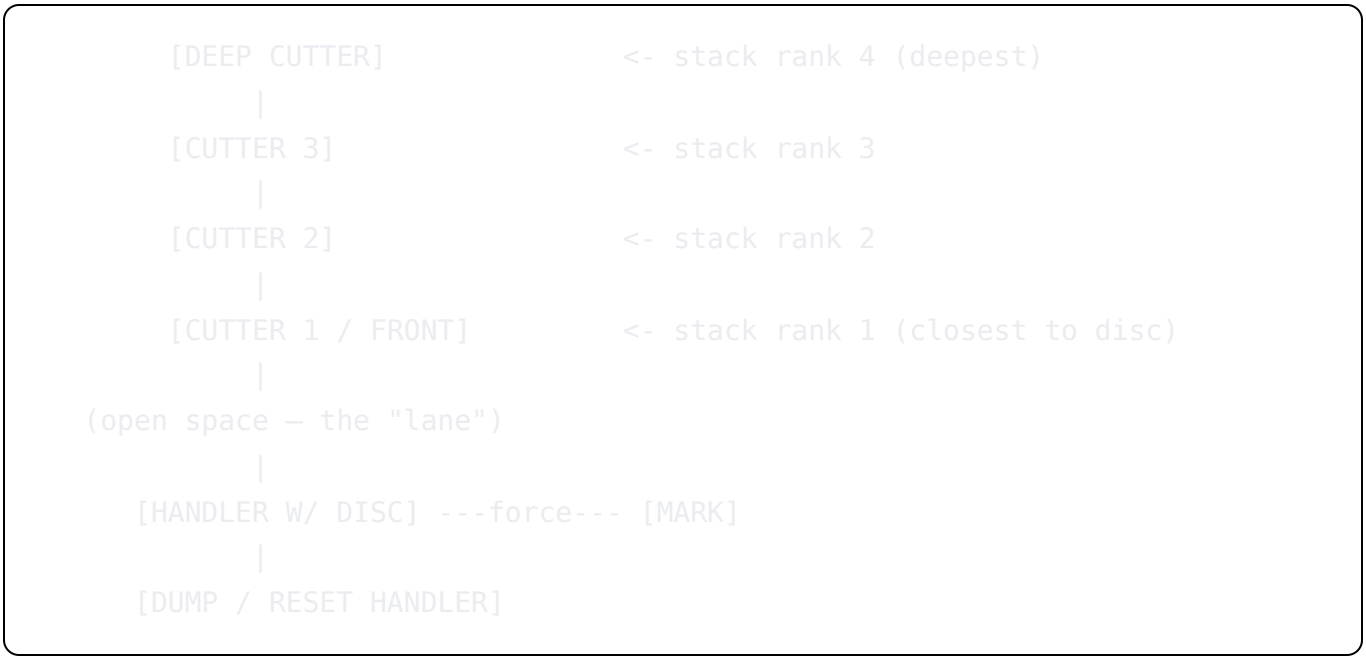
- **A fixed topology.** Every rep has the same skeleton: 1–2 handlers behind the disc, 4–5 cutters in a single file down the middle of the field. Football has 10 outfield players in unconstrained positions; vertical stack constrains 6–7 players to a near-linear arrangement.
- **An explicit rank order.** Cutters in the stack have a position — "first," "second," "third," deep — that coaches already name and reason about. This is a gift for graph learning: the ordering can be encoded directly as a feature instead of being something the model has to discover.
- **A repeating decision cycle.** Under-cut clears, next cutter goes, handler resets, repeat. This makes the prediction target ("who cuts next, and does it work") a naturally recurring, well-sampled event rather than a rare one.

The tradeoff: vertical stack is *one* offensive system among several (horizontal stack, split stack, zone offense, iso). A model trained on it will not generalize to those without retraining or an explicit system-classifier stage (Section 4). That's an acceptable scope limit for a first build.

---

2. Formation Anatomy (What the Graph Has to Represent)

A standard vertical stack, disc on the sideline-to-open-side axis, looks like this:



Key structural facts a model must be given, not left to infer from raw coordinates alone:

- **The stack occupies one lane** (typically the middle of the field, offset toward the break side), deliberately leaving the space on either side open for cuts.
- **Only the front 1–2 stack members are live threats at any instant** — deeper cutters are structurally "waiting their turn," which is exactly the kind of domain prior that raw trajectory data won't hand you for free.
- **The force (mark orientation) determines which half of the field is "open"** vs. "break," and every cut's value depends on which side it attacks relative to the force — this is the Ultimate analogue of TacticAI's marking-relationship edges, but directional rather than symmetric.

---

3. Graph Formalization

3.1 Node set — full 7v7, 15 nodes

Ultimate is played 7-on-7, so the graph should represent the **whole point**, not just the offense — the defense is half the information in the frame, exactly as it is in TacticAI (where all 22 football players are nodes, not just the attacking side). That gives **14 player nodes + 1 disc node = 15 nodes total**:

Node type	Count	Core features
Handler (w/ disc)	1	(x, y), stall count, force direction, throws remaining before forced pass
Reset / dump handler	1	(x, y), distance to thrower, marked (bool)
Stack cutter	4–5	(x, y), velocity vector, **stack rank (1 = front, N = deep)**, time since last cut, marked (bool)
Defender on thrower (the mark)	1	(x, y) relative to thrower, force-side being forced (forehand/backhand), reaction distance to thrower
Defender on each remaining offensive player	6	(x, y), velocity vector, marking leverage (force-side or break-side of assignment), man vs. help-defense flag
Disc	1	(x, y), possession state (grounded / in-hand / in-flight), z-height if in flight

That's 7 offense + 7 defense + 1 disc = **15 nodes**, matching TacticAI's convention of including the full defensive shape rather than treating markers as a derived property of the offense alone. All 14 player nodes share the same feature schema below regardless of side; only the edge structure (Section 3.2) differs by role.

Every player node also carries physical-attribute features, directly analogous to TacticAI including player height (used there for aerial-duel prediction on corners):

Feature	Why it matters in Ultimate
Height	Predicts contested-catch and layout-block ("sky") outcomes on 50/50 discs and end-zone jump-ball situations — the direct analogue of TacticAI's heading-duel use of height
Vertical leap	Refines the height signal for sky/block probability beyond standing height alone
Top speed / acceleration	Deep-cutter separation, layout-D range, handler under-cut closing speed
Wingspan / reach	Marking effectiveness (how much of the throwing lane a mark actually closes), block range on the mark
Fatigue proxy (distance covered this point, points played this game)	Completion and turnover rates measurably shift late in long points and late in tournaments; TacticAI's single-frame corner setting has no analogue to this, but Ultimate's longer live points do

These are static or slow-varying per-player attributes (roster-level, not per-frame), so they're collected once and joined onto every graph the player appears in — cheap signal relative to what it buys on 50/50-ball predictions specifically.

The **stack rank** feature remains the single biggest structural departure from TacticAI's node design. Football players have no canonical ordering, so TacticAI relies purely on permutation-invariant message passing across all 22 nodes. Vertical stack's offensive cutters *do* have a canonical order, so the architecture should exploit it rather than discard it (see Section 5) — while the 6 non-mark defenders stay unordered/permutation-invariant, tied to their assignment via marking edges rather than any rank of their own.

3.2 Edge set

Edge type	Connects	Represents
Stack-adjacency	rank i ↔ rank $i+1$	Physical spacing/timing coupling — a rank-2 cutter's window opens partly *because* rank-1 clears
Force/mark	thrower ↔ mark	Which throwing lanes are open vs. contested
Throw-lane visibility	thrower ↔ every cutter/handler	Binary or continuous "is there a clean throwing lane," computed geometrically (line-of-sight through defenders) rather than learned — a strong hand-engineered feature that reduces data burden

Edge type	Connects	Represents
Defender-marking	each offensive player ↔ their assigned defender node	Direct analogue of TacticAI's marking edge, now 1-to-1 across all 7 pairs rather than a distance heuristic, since defenders are full nodes with their own position/velocity/physical features
Help/switch	any defender ↔ any offensive player they are not primarily assigned to but are within intervention distance of	Captures zone/switch looks and poaches, which vertical-stack defenses use constantly against the deep cutter specifically
Reset-adjacency	thrower ↔ dump handler	Backfield continuation option, always available regardless of stack state

3.3 Graph-level features

Stall count, field zone (own end-zone / mid-field / red-zone), possession-direction (which end zone the team is attacking), and — genuinely underused in the football version — **wind**, which materially changes huck and break-throw success rates and is trivial to log per point.

3.4 Symmetry group (correcting the previous report's table)

Vertical stack is **not** left-right symmetric in the way a football pitch is: the stack sits on a lane relative to the sideline and the force direction, so a horizontal flip changes which side is "open" vs. "break," which is a semantically real difference, not a redundant view. The valid symmetry to exploit is narrower than TacticAI's four-way reflection group:

- **Valid:** reflecting the *entire play* (stack + force + defenders) end-to-end, since offense mechanics are direction-agnostic.
- **Invalid without relabeling:** reflecting only the field left-right while keeping the force label fixed — this actually changes the meaning of "open side," so a naive equivariant-CNN port from TacticAI would silently corrupt the open/break-side signal unless the force label is flipped in lockstep.

This is worth stating plainly because it's the kind of detail that looks like a copy-paste architecture choice but would quietly break the model if missed.

---

4. Prediction Targets

Ordered from easiest (best first deliverable) to hardest:

1. **Next-cutter identity** — given the current frozen frame (stack ranks, force, marks), which player receives the next completed pass. Direct analogue of TacticAI's receiver-prediction head.
2. **Cut-success / turnover probability** — for a given (thrower, target) pair, probability the throw is completed vs. blocked/dropped/intercepted. TacticAI's shot-probability head, re-purposed.
3. **Continuation-probability by lane** — probability of a positive-yardage completion split by open-side under, break-side under, and deep, since these are the three decisions a vertical-stack handler actually chooses between on every stall count.
4. **Generative placement suggestion** (stretch, Phase 3) — "if cutter rank 2 had cleared 1 second earlier, completion probability to rank 1 rises by X%." Direct analogue of TacticAI's generative tactic arm, but operating on *timing* offsets as much as spatial ones, since vertical stack failures are usually timing failures (two cutters occupying the same lane at once) rather than pure positioning failures.

---

5. Architecture: Why a Plain GNN Port Isn't Quite Right Here

TacticAI's permutation-invariant message passing is the correct choice for football because there is no canonical player ordering to exploit. Vertical stack *has* one (stack rank), so throwing it away and relying on the GNN to rediscover it from coordinates alone wastes the clearest signal in the data. Two credible designs:

A. GNN with rank as a node feature (simplest, recommended first). Standard message-passing GNN (`GATv2Conv` / `GINEConv` in PyTorch Geometric), but stack-rank is concatenated into each cutter node's feature vector, and stack-adjacency edges are given as a separate edge type from marking/lane edges (a heterogeneous graph, e.g. via `torch_geometric.nn.HeteroConv`). This keeps the equivariance benefits TacticAI relies on for the *unordered* parts of the graph (which defender marks whom) while giving the model explicit access to order for the part of the play that genuinely has one.

B. Hybrid: sequence encoder for the stack + GNN for everything else. Run the stack (ranked cutters) through a small positional-encoded transformer or 1D-conv sequence model to get a "stack state" embedding, then feed that embedding as a single graph-level feature alongside handler/mark/disc nodes into the GNN. This mirrors how a coach actually reasons about it — "read the stack as a unit, then decide" — and is worth an ablation against (A) once Phase 1 data exists, but adds complexity that isn't justified for a first build.

Recommendation: ship (A) first. It's a strictly smaller engineering delta from the original TacticAI codebase (PyTorch Geometric, same message-passing backbone) and gives a real baseline to compare (B) against later.

---

6. Data Pipeline (Vertical-Stack-Specific Additions)

The general CV pipeline from the earlier report (detection → tracking → disc detection → homography) is a prerequisite and is not repeated here. What's new for vertical stack specifically:

1. **Formation classifier (new stage).** Before any graph is built, a frame needs to be tagged as "vertical stack" vs. "horizontal stack" vs. "zone" vs. "transition/scramble." A simple approach: cluster offensive player positions per frame and check for a near-linear arrangement with low lateral spread — a k-means or PCA-based linearity score on the 6–7 offensive (x, y) points, thresholded, is enough for a first pass; a learned classifier (small MLP on positional summary stats) is a fallback if the heuristic is noisy.
2. **Stack-rank assignment.** Once a frame is classified as vertical stack, rank cutters by distance along the stack's principal axis (from the PCA step above), front-to-back. This needs temporal smoothing (a Kalman filter or simple exponential smoothing on rank-vs-time per tracked player ID) since raw per-frame ranking will jitter when two cutters are momentarily level with each other.
3. **Force/mark-side labeling.** Determine force direction from the mark's body orientation relative to the thrower — either a pose-based heuristic (shoulder-line angle from the keypoints already extracted for player detection) or manual annotation for the first dataset, since this is a low-volume, high-value label (one per throw, not one per frame).
4. **Event segmentation.** Slice continuous point footage into discrete "reps" — each rep starts when a cutter's movement pattern breaks from the "waiting in stack" baseline (a cut) and ends at catch/drop/block. This segmentation is what turns continuous video into the discrete (frozen-frame → outcome) examples that Section 5's architecture actually trains on, so it matters as much as the CV pipeline itself.

---

7. Phased Build Plan

Phase	Deliverable	Depends on
0	CV pipeline (detection, tracking, disc, homography) — reused from the general report, unchanged	Existing club/NCUC footage
1	Formation classifier + stack-rank assignment + rep segmentation	Phase 0
2	GNN (Section 5, design A) predicting next-cutter and completion probability on frozen vertical-stack	Phase 1, labeled reps

Phase	Deliverable	Depends on
	frames	
3	Lane-level continuation-probability head (open/break/deep split)	Phase 2, larger rep count
4 (stretch)	Generative timing-offset suggestions; hybrid sequence+GNN ablation (design B)	Phase 3, sufficient data volume to support two model families

This is deliberately narrower than the general report's four phases — everything here is scoped to one offensive system, which is what makes Phase 2 achievable on club-sized data rather than requiring pro-team data volume.

---

8. What Stays the Same From the General Report

To avoid duplicating the earlier document: the CV tooling choices (YOLOv8/9 + ByteTrack/BoT-SORT + custom disc detector), the AWS reference architecture (S3 → SageMaker Processing → SageMaker Training → Model Registry → Serverless Inference), and the core data-scarcity argument all carry over unchanged. Vertical stack doesn't change *how* you get tracking data — it changes what you do with it once you have it.

---

9. Bottom Line

Question	Answer
Is vertical stack a good first target vs. general Ultimate offense?	Yes — fixed topology, explicit rank order, and a naturally recurring decision cycle make it the most tractable slice of the sport to model
Does TacticAI's architecture port directly?	Mostly, with one real change: stack rank should be given to the model as an explicit feature/edge structure rather than discarded, since (unlike football) it isn't a permutation-invariant set
Is the symmetry group the same as football's?	No — full pitch reflection breaks the open/break-side force label unless flipped in lockstep; this is a genuine architectural difference, not a simplification
What's new in the data pipeline vs. the general report?	A formation classifier, stack-rank assignment with temporal smoothing, force/mark-side labeling, and rep segmentation — all upstream of the GNN itself

Question	Answer
Recommended entry point	Phase 0–1 (reuse existing CV pipeline, add stack-specific labeling), then Phase 2 (GNN with rank-aware heterogeneous graph) as the first real deliverable

References

1. Wang, Z., Veličković, P., Hennes, D. et al. *TacticAI: an AI assistant for football tactics*. **Nature Communications** 15, 1906 (2024). https://www.nature.com/articles/s41467-024-45965-x
2. arXiv preprint: https://arxiv.org/abs/2310.10553
3. DeepMind project overview: https://deepmind.google/blog/tacticai-ai-assistant-for-football-tactics/
4. Prior document: *Feasibility Report: A TacticAI-Style Tactical AI for Ultimate Frisbee* (general framing, CV pipeline, AWS architecture — referenced but not repeated above)